



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение

высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра математического обеспечения и стандартизации

информационных технологий (МОСИТ)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7

по дисциплине

«Структуры и алгоритмы обработки данных»

Тема: Линейные динамические списки

Выполнил студент группы ИКБО-64-23

Астахов С.П.

Принял старший преподаватель кафедры

Скворцова Л. А.

Москва 2024

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ И УСЛОВИЕ	3
2 РАЗРАБОТКА ПРОГРАММЫ.....	4
2.1 Реализация данных АДТ	4
2.1 Реализация алгоритма на языке C++ (1 задача)	6
2.2 Тестирование дополнительной операции Ошибка! Закладка не определена.	
2.3 Реализация алгоритма на языке C++ (2 задача) Ошибка! Закладка не определена.	
2.4 Тестирования дополнительной операции на vector	9
3 ВЫВОДЫ	12
4 СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ	20

1 ПОСТАНОВКА ЗАДАЧИ И УСЛОВИЕ

Цели работы:

- получение умений и навыков по созданию динамически формируемых структур данных средствами языка C++;
- получение навыков применения линейных списков в реализации алгоритмов.

Задачи:

1. Разработать и реализовать программу управлению линейным однонаправленным списком в соответствии с условием задачи, определенной вариантом.
2. Индивидуальный вариант №6 (задача 1): Дан линейный однонаправленный список L:
 - 1) Вставить перед последним узлом два новых узла с заданными значениями.
 - 2) Удалить из списка L первое отрицательное значение, если оно присутствует в списке.
 - 3) Найти в списке L максимальное значение и перенести узел с этим значением в конец списка.
3. Разработать и реализовать программу управлению линейным двунаправленным списком в соответствии с условием задачи, определенной вариантом.
4. Индивидуальный вариант №6 (задача 2): Автоматизированная система учета пациентов поликлиники хранит сведения о пациентах в двунаправленном списке. Вставить новые сведения по заболеванию конкретного пациента перед первым узлом с его мед. полисом, если данных в списке по нему еще нет, то узел вставить в конец списка

2 РАЗРАБОТКА ПРОГРАММЫ

2.1 Реализация данных АД

Для разработки решения 1 задачи (нашего АД) составим его описание, в котором будут определены основные поля, функции для выполнения алгоритма

АД List {

Переменные:

head - указатель на начало списка.

Size – количество узлов односвязного списка.

L - объект класса List, представляющие список L соответственно.

T – какой либо тип данных.

Функции:

List () {

конструктор, инициализирует голову списка как nullptr.

}

~List () {

Деструктор, отчищает динамическую память, чтоб не было утечки
памяти

}

GetSize(){

Метод-геттер, с помощью которого мы получаем переменную Size

```
}
```

```
Pop_front(){
```

Метод, удаляющий первый узел списка

```
}
```

```
push_front(T data){
```

Метод, вставки в начало

```
}
```

```
creat_list(int n){
```

Создает список из n узлов

```
}
```

```
Opetator[](const int index){
```

Метод-оператор, позволяющий получать узлы списка через квадратные скобочки

```
}
```

```
task1(T a, T b) {
```

Метод, 2 вставки узлов перед последним узлом с заданными значениями

```
}
```

```
task2(){
```

Метод, удаление из списка L первое отрицательное значение, если оно присутствует в списке

```
}
```

```
Task3(){
```

Метод нахождения в списке L максимальное значение и перенести узел с этим значением в конец списка.

```
}
```

2.2 Реализация алгоритма на языке C++ (1 задача)

Header файл(List(1 зад).h)

```
#pragma once
```

```
#include <iostream>
```

```
using std::cout;
```

```
using std::cin;
```

```
using std::endl;
```

```
template<typename T>
```

```
class List{
```

```
public:
```

```
List();
```

```
~List();
```

```
T& operator[](const int index);
```

```
int GetSize() { return Size; }; // Получение размера
```

```
void pop_front(); // Удаление первого элемента
```

```
void push_front(T data);
```

```
void creat_list(int n);
```

```
void task1(T a, T b);
```

```
void task2();
```

```
void task3();
```

```
private:
```

```
template<typename T>
```

```
class Node {
```

```
public:
```

```
Node* pNext;
```

```
T data;
```

```

        Node(T data = T(), Node* pNext = nullptr) {
            this->data = data;
            this->pNext = pNext;
        }
};
int Size;
Node<T> *head;

};

// В отдельный файл нельзя вынести шаблонные методы класса

template<typename T>
List<T>::List() {
    Size = 0;
    head = nullptr;
}

template<typename T>
List<T>::~~List() {
    while (Size) {
        pop_front();
    }
}

template<typename T>
T& List<T>::operator[](const int index) {
    int counter = 0;
    Node<T>* current = this->head;
    while (current != nullptr) {

        if (counter == index)
        {
            return current->data;
        }

        current = current->pNext;
        counter++;
    }
}

template<typename T>
void List<T>::pop_front() {
    Node<T>* temp = head;
    head = head->pNext;
    delete temp;
    Size--;
}

template<typename T>
void List<T>::push_front(T data) {
    head = new Node<T>(data, head);
    Size++;
}

```

```

template<typename T>
void List<T>::creat_list(int n){
    T data;

    for (int i = 0; i < n; i++) {
        cout << "Введите данные: "; cin >> data;
        push_front(data);
    }
}

template<typename T>
void List<T>::task1(T a, T b) {
    Node<T>* current = head;

    if (current == nullptr || current->pNext == nullptr) {
        return;
    }

    while (current->pNext->pNext != nullptr) {
        current = current->pNext;
    }

    current->pNext = new Node<T>(a, current->pNext);
    current->pNext->pNext = new Node<T>(b, current->pNext->pNext);
    Size += 2;
}

template<typename T>
void List<T>::task2() {
    Node<T>* current = head;
    Node<T>* prev = nullptr;
    while (current != nullptr && current->data >= 0) {
        prev = current;
        current = current->pNext;
    }

    if (current != nullptr && prev != nullptr) {
        prev->pNext = current->pNext;
        delete current;
        Size--;
    }
    else if (current != nullptr) {
        pop_front();
    }
}

template<typename T>
void List<T>::task3() {
    if (head == nullptr || head->pNext == nullptr) {
        return; // List is empty or has only one node.
    }
    Node<T>* current = head;
    Node<T>* maxNodePrev = nullptr;
    Node<T>* maxNode = head;
}

```



```

Node<T>* prev = nullptr;
T maxValue = head->data;

while (current != nullptr) {
    if (current->data > maxValue) {
        maxValue = current->data;
        maxNodePrev = prev;
        maxNode = current;
    }
    prev = current;
    current = current->pNext;
}

if (maxNode->pNext == nullptr) {
    return;
}

if (maxNodePrev == nullptr) {
    head = maxNode->pNext;
}
else {
    maxNodePrev->pNext = maxNode->pNext;
}

prev->pNext = maxNode;
maxNode->pNext = nullptr;
}

```

Сpp файл (7 работа.cpp)

```

#include "List(1 зад).h"

int main(){
    setlocale(0, "Rus");
    List<double> L;
    int choice = 1;
    double a, b;
    // 4-6 Пункты доп задачи
    while (choice != 0) {
        puts("Интерфейс");
        puts("1) Вставка узла в начало списка");
        puts("2) Создание списка из n узлов, используя функцию вставки узла перед первым узлом");
        puts("3) Вывод списка в консоль");
        puts("4) Вставить перед последним узлом два новых узла с заданными значениями");
        puts("5) Удалить из списка L первое отрицательное значение, если оно присутствует в списке");
        puts("6) Найти в списке L максимальное значение и перенести узел с этим значением в конец списка");
        puts("0) Чтоб выйти");
        cout << "Ввод: "; cin >> choice;

        switch (choice) {
            case 1:
                int data;

```

```

        cout << "Введите данные: "; cin >> data;
        L.push_front(data);
        break;
    case 2:
        int n;
        cout << "Введите колл-во(n) элементов списка для создания: "; cin
>> n;

        L.creat_list(n);
        break;
    case 3:
        for (int i = 0; i < L.GetSize(); i++) {
            cout << L[i] << " ";
        }
        cout << endl;
        break;
    case 4:
        cout << "Введите два значения для вставки в предпоследний узел(че-
рез пробел): ";
        cin >> a >> b;
        L.task1(a, b);
        break;
    case 5:
        L.task2();
        break;
    case 6:
        L.task3();
        break;
    }
}
}

```

2.3 Тестирование

```

Интерфейс
1) Вставка узла в начало списка
2) Создание списка из n узлов, используя функцию вставки узла перед первым узлом
3) Вывод списка в консоль
4) Вставить перед последним узлом два новых узла с заданными значениями
5) Удалить из списка L первое отрицательное значение, если оно присутствует в списке
6) Найти в списке L максимальное значение и перенести узел с этим значением в конец списка
0) Чтоб выйти
Ввод: 1
Введите данные: 3
Интерфейс
1) Вставка узла в начало списка
2) Создание списка из n узлов, используя функцию вставки узла перед первым узлом
3) Вывод списка в консоль
4) Вставить перед последним узлом два новых узла с заданными значениями
5) Удалить из списка L первое отрицательное значение, если оно присутствует в списке
6) Найти в списке L максимальное значение и перенести узел с этим значением в конец списка
0) Чтоб выйти
Ввод: 3
3

```

Рис.1

```

Ввод: 2
Введите колл-во(n) элементов списка для создания: 5
Введите данные: 1
Введите данные: 2
Введите данные: 3
Введите данные: 4
Введите данные: 5
Интерфейс
1) Вставка узла в начало списка
2) Создание списка из n узлов, используя функцию вставки узла перед первым узлом
3) Вывод списка в консоль
4) Вставить перед последним узлом два новых узла с заданными значениями
5) Удалить из списка L первое отрицательное значение, если оно присутствует в списке
6) Найти в списке L максимальное значение и перенести узел с этим значением в конец списка
0) Чтоб выйти
Ввод: 4
Введите два значения для вставки в предпоследний узел(через пробел): 24 48
Интерфейс
1) Вставка узла в начало списка
2) Создание списка из n узлов, используя функцию вставки узла перед первым узлом
3) Вывод списка в консоль
4) Вставить перед последним узлом два новых узла с заданными значениями
5) Удалить из списка L первое отрицательное значение, если оно присутствует в списке
6) Найти в списке L максимальное значение и перенести узел с этим значением в конец списка
0) Чтоб выйти
Ввод: 3
5 4 3 2 1 24 48 3

```

Рис.2

Все результаты совпали с ожидаемыми, что говорит о правильности программы.

2.4 Реализация данных АД

Для разработки решения 2 задачи (нашего АД) составим его описание, в котором будут определены основные поля, функции для выполнения алгоритма

```
АД PatientInfo{  
    Переменные:  
        String insuranceNumber;  
        String visitDate;  
        int diagnosisCode;  
  
    Метод:  
        PatientInfo(string insNumber, string date, int code){  
            Конструктор, для определения значений пациента  
        }  
}  
  
АД List{  
    АД Node{  
        Переменные:  
            PatientInfo data;  
            Node* prev;  
            Node* next;  
  
        Метод:  
            Node(){
```

Конструктор

```
}
```

```
}
```

Переменные:

```
Node* head;
```

```
Node* tail;
```

```
List(){Конструктор}
```

```
~List(){Деструктор}
```

```
insertFront(PatientInfo info) {метод Вставка узла перед первым узлом};
```

```
createList(int n) {Метод Создание списка из n узлов};
```

```
printForward() const { Метод Вывод списка справа налево}
```

```
printBackward() const { Метод Вывод списка слово направо}
```

```
find(string insNumber){метод Поиск узла с заданным значением ключа}
```

```
insertPatientInfo(PatientInfo info) {Метод Вставка новых сведений о  
заболевании пациента}
```

```
}
```

2.5 Реализация алгоритма на языке C++ (2 задача)

```
Header файл( List(2 зад).h )
#include <iostream>
#include <string>
using std::cout;
using std::cin;
using std::endl;
// Структура для хранения информации о пациенте
struct PatientInfo {
    std::string insuranceNumber; // Номер медицинского полиса
    std::string visitDate;      // Дата обращения
    int diagnosisCode;          // Код диагноза

    PatientInfo(std::string insNumber, std::string date, int code)
        : insuranceNumber(insNumber), visitDate(date), diagnosisCode(code) {}
};

// Класс для управления двунаправленным списком
class List {
public:
    struct Node {
        PatientInfo data;
        Node* prev;
        Node* next;
        Node(PatientInfo val) : data(val), prev(nullptr), next(nullptr) {}
    };

    List() : head(nullptr), tail(nullptr) {}

    ~List() {
        Node* current = head;
        while (current != nullptr) {
            Node* next = current->next;
            delete current;
            current = next;
        }
    }

    // Вставка узла перед первым узлом
    void insertFront(PatientInfo info) {
        Node* newNode = new Node(info);
        if (head == nullptr) {
            head = tail = newNode;
        }
        else {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
        }
    }

    // Создание списка из n узлов
```

```

void createList(int n) {
    std::string insNumber, date; int code;

    for (int i = 0; i < n; ++i) {
        cout << i+1 << ")" << endl;
        cout << "Полис: "; cin >> insNumber;
        cout << "Дата: "; cin >> date;
        cout << "Код диаг.: "; cin >> code;

        insertFront(PatientInfo(insNumber, date, code));
    }
}

// Вывод списка слева направо
void printForward() const {
    Node* current = head;
    while (current != nullptr) {
        std::cout << "Номер полиса: " << current->data.insuranceNumber
            << ", Дата обращения: " << current->data.visitDate
            << ", Код диагноза: " << current->data.diagnosisCode << std::endl;
        current = current->next;
    }
}

// Вывод списка справа налево
void printBackward() const {
    Node* current = tail;
    while (current != nullptr) {
        std::cout << "Номер полиса: " << current->data.insuranceNumber
            << ", Дата обращения: " << current->data.visitDate
            << ", Код диагноза: " << current->data.diagnosisCode << std::endl;
        current = current->prev;
    }
}

// Поиск узла с заданным значением ключа
Node* find(std::string insNumber) const {
    Node* current = head;
    while (current != nullptr && current->data.insuranceNumber != insNumber) {
        current = current->next;
    }
    return current; // Возвращает nullptr, если узел не найден
}

// Вставка новых сведений о заболевании пациента
void insertPatientInfo(PatientInfo info) {
    Node* existingNode = find(info.insuranceNumber);
    if (existingNode != nullptr) {
        // Вставляем перед первым узлом с мед. полисом
        Node* newNode = new Node(info);
        newNode->next = existingNode;
        newNode->prev = existingNode->prev;
        if (existingNode->prev != nullptr) {
            existingNode->prev->next = newNode;
        }
    }
}

```

```

    }
    else {
        head = newNode;
    }
    existingNode->prev = newNode;
}
else {
    // Вставляем в конец списка, если узел не найден
    Node* newNode = new Node(info);
    if (tail != nullptr) {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
    else {
        head = tail = newNode;
    }
}
}

private:
    Node* head;
    Node* tail;
};

```

Сpp файл

```

#include "List(2 зад).h"

int main() {
    setlocale(0, "Rus");
    List L;
    int choice = 1;

    std::string medPolis;
    std::string date;
    int diagnosisCode;

    std::string value;
    int n;
    // 4-6 Пункты доп задачи
    while (choice != 0) {
        puts("Интерфейс");
        puts("1) Создание списка из n узлов, используя функцию вставки узла перед первым узлом");
        puts("2) Вывод списка в консоль");
        puts("3) Поиск узла с заданным значением ключа");
        puts("4) Вставить новые сведения по заболеванию конкретного пациента перед первым узлом\n\");
        с его мед. полисом, если данных в списке по нему еще нет, то узел вставить в конец списка.");
        puts("0) Чтоб выйти");
        cout << "Ввод: "; cin >> choice;
    }
}

```



```

switch (choice) {
case 1:
    cout << "Сколько n узлов надо создать? "; cin >> n;
    L.createList(n);
    break;
case 2:
    int a;
    puts("Как вывести?");
    puts("1) Слево направо");
    puts("2) Справо налево");
    cout << "Выбор-> "; cin >> a;
    if (a == 1) {
        L.printForward();
    }
    else if (a == 2) {
        L.printBackward();
    }
    break;
case 3:
    cout << "Кого нужно найти по полису?" << endl;
    cin >> value;
    if (L.find(value) != nullptr)
        cout << L.find(value) << endl;
    else
        cout << "Не найдено!" << endl;
    break;
case 4:

    cout << "Номер полиса: "; cin >> medPolis;
    cout << "Дата обращения: "; cin >> date;
    cout << "Код диагноза: "; cin >> diagnosisCode;

    L.insertPatientInfo(PatientInfo(medPolis, date, diagnosisCode));
    break;
}
}
}

```

2.6 Тестирования

```
Интерфейс
1) Создание списка из n узлов, используя функцию вставки узла перед первым узлом
2) Вывод списка в консоль
3) Поиск узла с заданным значением ключа
4) Вставить новые сведения по заболеванию конкретного пациента перед первым узлом
с его мед. полисом, если данных в списке по нему еще нет, то узел вставить в конец списка.
0) Чтоб выйти
Ввод: 1
Сколько n узлов надо создать? 3
1)
Полис: 123-000-24-23
Дата: 2024-23-04
Код диаг.: 1
2)
Полис: 134-553-89-00
Дата: 2023-30-12
Код диаг.: 2
3)
Полис: 111-222-43-55
Дата: 2000-2
Код диаг.:
3
```

Рис. 3

```
Интерфейс
1) Создание списка из n узлов, используя функцию вставки узла перед первым узлом
2) Вывод списка в консоль
3) Поиск узла с заданным значением ключа
4) Вставить новые сведения по заболеванию конкретного пациента перед первым узлом
с его мед. полисом, если данных в списке по нему еще нет, то узел вставить в конец списка.
0) Чтоб выйти
Ввод: 2
Как вывести?
1) Слево направо
2) Справо налево
Выбор-> 1
Номер полиса: 111-222-43-55, Дата обращения: 2000-2, Код диагноза: 3
Номер полиса: 134-553-89-00, Дата обращения: 2023-30-12, Код диагноза: 2
Номер полиса: 123-000-24-23, Дата обращения: 2024-23-04, Код диагноза: 1
```

Рис. 4

Все результаты совпали с ожидаемыми, что говорит о правильности программы.

3 ВЫВОДЫ

В результате изучения языка C++ и применения линейных списков для реализации алгоритмов структур данных были приобретены не только технические навыки, но и понимание принципов динамического формирования структур данных. Это позволяет эффективно создавать и использовать динамические структуры данных в программах на языке C++, повышая их гибкость и эффективность.

4 СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1) Скворцова Л.А, Гусев К.В, Трушин С.М, Филатов А.С: Учебно-методическое пособие СиАОД часть 1 / — МИРЭА — Российский технологический университет, 2021. – 146 с

2) Лекционный материал Скворцовой Л.А (старший преподаватель кафедры МОСИТ)